TECHNICAL DESIGN DOCUMENT — v1.0

Cybercrit: Implementation Roadmap

A freemium, hybrid-analysis CLI for git pre-commit security review. Zero-repo-scan architecture. Phase 1: local static analysis. Phase 2: cloud LLM on diff delta only.

Target Language **Go 1.22+** Hook Trigger **git pre-commit** Phase 1 Budget **< 80ms**

Phase 2 Budget **< 3s (P95)** Diff Input **stdin | git diff --cached**

CONTENTS

§1 Tech Stack Selection

§2 Execution Flow (The Git Hook)

§3 Data Schemas (Prompting & Parsing)

§4 Phased Development Milestones

§5 Edge Cases & Mitigations

§ 01 Tech Stack Selection: Go vs. Rust

VERDICT: GO

SELECTED

Go 1.22+

Cold startup	~5ms
Cross-compile	GOOS/GOARCH trivial
Binary size	~4MB stripped
CGO for Semgrep?	Not needed (subprocess)
JSON handling	encoding/json native
HTTP client	net/http stdlib
Dev iteration	Fast
Git subprocess	os/exec clean

Rust

Cold startup	~2ms
Cross-compile	cross toolchain needed
Binary size	~1.5MB stripped
CGO equivalent	FFI verbose
JSON handling	serde + derive macros
HTTP client	reqwest (async tokio)
Dev iteration	Slow compile cycles
Git subprocess	std::process verbose

DECISION RATIONALE

The 3ms startup advantage Rust offers is irrelevant against the network I/O dominance of Phase 2 (Groq API call). Go's `GOOS=darwin/linux/windows GOARCH=amd64/arm64` cross-compilation in a single CI step with zero native deps is the decisive factor for a freemium distribution model. Rust's borrow checker adds meaningful friction when iterating on the diff-parsing and interactive terminal logic — Go's goroutines handle

Phase 1/2 parallel execution more readably. Use `goreleaser` for automated multi-platform binary releases + SHA256 checksums. Single static binary: no JVM, no Python venv, no Node modules.

project layout

```
cybercrit/
├── cmd/cybercrit/ # main entrypoint + flag parsing
├── internal/
│ ├── diff/ # git diff extraction + parsing
│ ├── analyzer/ # semgrep subprocess wrapper
│ ├── ai/ # LLM client (Groq / OpenRouter)
│ ├── config/ # BYOK key resolution + .cybercrit.toml
│ ├── tui/ # bubbletea interactive prompts
│ └── patcher/ # unified diff apply logic
├── .goreleaser.yaml
└── install.sh # hooks into .git/hooks/pre-commit
```

§ 02 Execution Flow: From Enter to Pass/Fail

TIMING ANNOTATED

T+0 **Developer hits Enter on `git commit -m "..."`**

Git's hook dispatcher reads `.git/hooks/pre-commit` — a shell script that `exec cybercrit hook`. The hook script has the cybercrit binary path resolved at install time, stored absolutely. Git pauses all commit logic until exit code is returned.

`0ms`

T+1 **Binary boots + captures diff**

Process starts. Config loaded from `$XDG_CONFIG_HOME/cybercrit/config.toml` (memory-mapped). Executes `git diff --cached --unified=3 --no-color` via `os/exec`, capturing stdout into a byte buffer. Diff is parsed into a typed `[]FileDiff` struct — file path, hunk headers, added/removed line arrays, language detected by extension.

`~8ms total`

T+2 **Phase 1: Local static analysis (Semgrep subprocess)**

Writes added lines only to a temp file in `os.TempDir()` using the correct file extension. Launches `semgrep --config auto --json --quiet <tempfile>` with a 2s hard timeout context. Parses JSON output into `[]Finding`. On timeout or semgrep-not-found: logs warning, skips Phase 1, proceeds to Phase 2. Temp file deleted via `defer os.Remove()`. Semgrep runs in-process with rules loaded from `~/.semgrep/` (pre-cached on install).

`~40-70ms`

T+3 **Severity gate: should AI Phase run?**

If Phase 1 returns CRITICAL/HIGH findings with `confidence: HIGH`: block commit immediately (exit 1) and print interactive fix prompt — no tokens consumed. If findings are LOW/MEDIUM or Phase 1 clean AND diff complexity score exceeds threshold (cyclomatic proxy: line count × file count > N, default 50): route to Phase 2. If diff is trivial (comment changes, whitespace): `exit 0` silently.

~1ms

T+4 **Phase 2: LLM API call (async, streamed)**

Constructs request payload using the structured system prompt (see §3). Strips diff to `added_lines` only — removes `-` prefixed lines to minimize tokens. Sends POST to Groq `/openai/v1/chat/completions` with `stream: true` and `response_format: {type: "json_object"}`. Streams response via SSE reader. A spinner is printed to stderr (not stdout) so git output is clean. 5s hard timeout via `context.WithTimeout`.

~800ms–2.5s

T+5 **Response parsed + interactive TUI rendered**

Full JSON response assembled from stream chunks. Unmarshalled into `AnalysisResult` schema. Each finding printed with: diff context (highlighted added lines), severity badge, vulnerability class, and confidence. Bubbletea TUI prompts: `[Y] Apply patch [N] Skip [I] Ignore rule [A] Abort commit`. Patches stored as unified diffs in the response schema and applied via the `patcher` package using `git apply --cached --index`.

interactive

T+6 **Commit passes or fails**

If all HIGH/CRITICAL findings resolved (fixed or explicitly ignored with `cybercrit-ignore` annotation added to code): `exit 0` → git proceeds. If user selects `[A]` or unfixed HIGH finding remains: `exit 1` → git aborts commit. Findings logged to `~/.cybercrit/audit.jsonl` for telemetry (opt-in).

~1ms

§ 03 Data Schemas: System Prompt & LLM Response

JSON SCHEMA

LLM System Prompt (exact string, injected at request construction) SYSTEM ROLE

```
You are a security-focused code auditor. You will be given a unified git diff containing only the ADDED lines (+) from a developer's staged commit. Your task is to identify real, exploitable vulnerabilities — not style issues, not theoretical risks. Rules: 1. Analyze ONLY the added lines. Do not comment on removed lines. 2. Only report findings with confidence ≥ 0.7. 3. Each finding MUST include a concrete, minimal patch as a unified diff string. 4. The patch MUST be directly applicable via `git apply`. It must contain correct hunk headers. 5. Do NOT hallucinate line numbers. The line_number field must reference an actual + line from the input diff. 6. Severity levels: CRITICAL | HIGH | MEDIUM | LOW. Only flag MEDIUM+ findings. 7. Vulnerability classes (use exactly): INJECTION | SECRETS | IDOR | SSRF | RCE | AUTH_BYPASS | CRYPTO_WEAK | PATH_TRAVERSAL | DESERIALIZATION | XXE | OPEN_REDIRECT 8. If no vulnerabilities are found, return an empty findings array. Do not fabricate findings. RESPOND ONLY WITH VALID JSON MATCHING THE PROVIDED SCHEMA. NO PREAMBLE. NO EXPLANATION OUTSIDE JSON.
```

User Message Template (constructed per-request) USER TURN

```
Analyze this diff for security vulnerabilities: ```diff {{.DiffContent}} ``` Language: {{.DetectedLanguage}} File: {{.FilePath}} Added line count: {{.AddedLineCount}}
```

Enforced JSON Response Schema (Go struct + JSON) RESPONSE_FORMAT: JSON_OBJECT

```
// Go type definitions type AnalysisResult struct { Findings []Finding `json:"findings"` TokensUsed int `json:"tokens_used,omitempty"` } type Finding struct { ID string `json:"id"` // sha256(file+line+class)[:8] Severity string `json:"severity"` // CRITICAL|HIGH|MEDIUM|LOW VulnClass string `json:"vuln_class"` // enum above Description string `json:"description"` // max 120 chars LineNumber int `json:"line_number"` // line in the ORIGINAL file CodeSnippet string `json:"code_snippet"` // verbatim offending line Confidence float64 `json:"confidence"` // 0.0 - 1.0 Fix Fix `json:"fix"` CWE string `json:"cwe"` // e.g. "CWE-89" } type Fix struct { UnifiedDiff string `json:"unified_diff"` // git apply compatible Explanation string `json:"explanation"` // max 200 chars, human readable Automated bool `json:"automated"` // false if manual review required }
```

```
// Example LLM response { "findings": [{ "id": "a3f9c1b2", "severity": "CRITICAL", "vuln_class": "INJECTION", "description": "User input directly interpolated into SQL query string", "line_number": 47, "code_snippet": "db.Query(\"SELECT * FROM users WHERE id=\" + userId)", "confidence": 0.97, "cwe": "CWE-89", "fix": { "unified_diff": "--- a/db/users.go\n+++ b/db/users.go\n@@ -47,1 +47,1 @@\n-db.Query(\"SELECT * FROM users WHERE id=\" + userId)\n+db.Query(\"SELECT * FROM users WHERE id=?\", userId)", "explanation": "Replace string concatenation with parameterized query to prevent SQL injection", "automated": true } }] }
```

The `response_format: {type: "json_object"}` parameter on the Groq API forces the model into JSON mode — it will never emit preamble text. The `unified_diff` field is validated post-parse with a custom Go regex before it is passed to `git apply` — any patch that fails the hunk header regex is flagged as non-automated and the user is shown the explanation only.

§ 04 Phased Development Milestones

4-PHASE BUILD

M1 **CLI Skeleton + Hook Infrastructure**
Target: working hook that captures and prints diffs — zero analysis

- **cybercrit install:** writes shell wrapper to `.git/hooks/pre-commit`, makes it executable, backs up any existing hook
- **cybercrit uninstall:** restores original hook, cleans up binary symlink
- **Diff engine:** `internal/diff` package — executes `git diff --cached`, parses unified diff format into `[]FileDiff{Path, Hunks, AddedLines, RemovedLines}`, language detection by extension map
- **Config system:** `.cybercrit.toml` in repo root (per-project) + `~/.config/cybercrit/config.toml` (global). Keys: `api_key`, `model`, `phase2_threshold`, `ignore_rules []string`
- **Empty-diff fast path:** if `len(addedLines) == 0`, exit 0 immediately with no output
- **goreleaser config:** builds for linux/darwin/windows × amd64/arm64 in CI

```
exit code: 0 always    boot + diff capture < 15ms    zero external deps at runtime
```

M2 Phase 1: Local Static Analysis Engine

Target: sub-80ms semgrep scan, deterministic block on HIGH findings

- **Semgrep wrapper:** `internal/analyzer` writes added-lines-only temp file, execs `semgrep --config p/secrets --config p/owasp-top-ten --json --quiet` with `context.WithTimeout(ctx, 2s)`
- **Graceful degradation:** if semgrep binary not found in PATH, skip Phase 1 entirely — log at DEBUG level, never user-visible error
- **Finding deduplication:** hash on `rule_id + line_content` to collapse identical findings across multiple hunks
- **Non-interactive output:** for CI environments (`$CI=true` or no TTY): print findings as machine-readable JSON to stderr, exit 1 on CRITICAL/HIGH. No interactive prompt.
- **Suppression annotations:** parse `// cybercrit-ignore: RULE_ID` comments in added lines → remove those findings before severity gate

```
phase 1 p95 < 80ms    CRITICAL exits 1 without AI call    CI-safe non-interactive mode
```

M3 Phase 2: LLM Integration + BYOK

Target: working Groq API call, JSON response parsed, finding printed to terminal

- **LLM client:** `internal/ai` package — OpenAI-compatible client targeting `https://api.groq.com/openai/v1/chat/completions`. SSE streaming reader. Configurable via `base_url` for OpenRouter/local Ollama fallback.
- **BYOK key resolution chain:** (1) `CYBERCRIT_API_KEY` env var → (2) keychain via `99designs/keychain` → (3) `config.toml` → (4) prompt for key on first run, store in OS keychain
- **Token budget enforcement:** diff truncated to 4096 tokens max before sending. Truncation is hunk-aware: drops lowest-complexity hunks first (measured by cyclomatic proxy), preserves file context lines.
- **Response validation:** JSON unmarshalled into `AnalysisResult`. Confidence threshold filter (`< 0.7` filtered). Line number cross-validated against actual diff lines — mismatched line numbers set `Fix.Automated = false`.

→ **Premium routing:** if `subscription_token` present in config, route via `https://api.cybercrit.dev/v1/analyze` instead of direct BYOK

phase 2 p95 < 3s zero token waste on ignored files BYOK fully functional

M4 Interactive TUI + Automated Patch Apply

Target: bubbletea-powered interactive fix workflow, git apply integration

→ **Bubbletea TUI:** renders each finding sequentially. Colored diff view (lipgloss): added lines in green, removed in red. Severity badge coloring. CWE + description. Per-finding keypress: `Y`=apply fix, `N`=skip (allow), `I`=permanently ignore rule (appends to config), `A`=abort commit

→ **Patch apply pipeline:** `internal/patcher` writes unified diff from `Fix.UnifiedDiff` to temp file, runs `git apply --cached --index <patchfile>`. On failure: surfaces error, sets `Fix.Automated=false` for that finding, shows manual instruction.

→ **Audit log:** appends JSONL record to `~/.cybercrit/audit.jsonl` on every finding — timestamps, finding ID, user action taken, file path, repo remote URL hash (anonymized)

→ **cybercrit review:** standalone subcommand for on-demand scan outside hook context. Accepts `--file`, `--since <commit>`, or piped diff on stdin.

full UX loop functional git apply success rate > 90% audit log structured JSONL

§ 05 Edge Cases & Architectural Mitigations

3 CRITICAL HURDLES

△ Hurdle 1: Massive Diffs (Schema Migrations, Generated Code, Lockfiles)

HIGH IMPACT

Problem: A `yarn.lock` update or a DB migration with 3,000 added lines will blast the context window, consume enormous tokens, and produce zero actionable security findings. The hook will appear to hang (2-8s API call) for a non-security-relevant change. This destroys developer trust faster than any false positive.

Root cause: Naive diff routing treats all file types identically.

> MITIGATION ARCHITECTURE
>
> Three-layer filtration before Phase 2: (1) **Extension blocklist** in config defaults: `skip_extensions = [".lock", ".sum", ".snap", "-lock.json", ".pb.go", "_gen.go", ".min.js"]` — these files never reach Phase 2. (2) **Hunk complexity scoring:** score = `added_lines × (unique_tokens / total_tokens)`. Purely additive, low-entropy hunks (e.g., sequential integer IDs in a migration) score near 0 and are dropped. (3) **Hard token cap with smart truncation:** if remaining diff exceeds 3,500 tokens, apply a priority queue — sort hunks by complexity score descending, fill to 3,500 tokens, append a `[DIFF TRUNCATED: N hunks omitted]` note in the user message. The LLM is explicitly told in the system prompt that truncation may have occurred.

Hurdle 2: API Rate Limits & Network Failure During Pre-commit

HIGH IMPACT

Problem: A pre-commit hook that blocks the commit because the Groq API returned 429 or the developer is on an airplane will be uninstalled within 24 hours. Any non-deterministic network dependency that can silently block developer workflow is existential for adoption.

Root cause: Treating Phase 2 as a hard gate rather than an advisory layer.

> MITIGATION ARCHITECTURE
>
> Phase 2 is **never a hard block by default**. The failure policy is configurable: `phase2_failure_policy = "warn"` (default) | `"block"` | `"skip"`. Under `"warn"`: API errors print a yellow warning, commit proceeds. Under `"block"`: commit aborts on API failure (for security-critical teams). Hard timeout is 5s via `context.WithDeadline`. On 429: exponential backoff with a **single** retry (500ms). On second failure: fail-open with warning. The response is cached by `sha256(diff_content)` to `~/.cybercrit/cache/` (TTL: 1 hour) — identical diffs on amended commits skip the API call entirely. Cache eviction is LRU with a 50MB cap managed via `bbolt`.

Hurdle 3: False Positives Eroding Developer Trust

MED IMPACT / HIGH FREQUENCY

Problem: LLMs hallucinate vulnerabilities. Semgrep rules have false positive rates of 15–30% on context-poor diffs. A developer who gets flagged for a non-issue twice will ignore all future alerts — defeating the tool's entire purpose. The patch applier applying an incorrect fix is worse than doing nothing.

Root cause: Both analysis phases lack feedback loops and confidence calibration against repo-specific context.

> MITIGATION ARCHITECTURE
>
> Four mechanisms: (1) **Confidence floor enforcement**: LLM findings below 0.70 confidence are silently dropped — never shown to the user. Between 0.70–0.85: shown as advisory, cannot block commit. Only >0.85 confidence findings can trigger a blocking prompt. (2) **Local ignore ledger**: when user presses `[I]`, the `rule_id + file_path_glob` pair is written to `.cybercrit.toml`'s `[[ignore]]` table. Same finding is never surfaced in that repo again. (3) **Patch dry-run gate**: `git apply --check` is always run before `git apply --cached`. If `--check` fails, the `[Y] Apply` option is grayed out and replaced with `[V] View diff only`. No broken patches ever touch the index. (4) **Semgrep autofix validation**: when semgrep provides its own `fix` field in JSON output, prefer it over the LLM-generated patch — semgrep fixes are rule-derived and deterministic.

KEY DEPENDENCIES

TUI	`charmbracelet/bubbletea`	Styling	`charmbracelet/lipgloss`	Config	`BurntSushi/toml`
Keychain	`99designs/keychain`	Cache DB	`etcd-io/bbolt`	Diff parse	`sourcegraph/go-`

diff

Releases

goreleaser/goreleaser

Static analysis semgrep

(subprocess)